GIT ASSIGNMENT-2

Git Branching Assignment

Create a new Git repository called `git-assignment` and initialize it using Git. Inside the repository, create a file called `app.txt` and add the text “DevOps Training” into the file. Add the file into the staging area and create the first commit with the message “Initial commit”. After creating the commit, verify the commit history using `git log --oneline`. Next, create a new branch called `feature-login` using the modern Git command. Switch into the branch and add a new line called “Login feature added” into `app.txt`. Commit the changes with the message “Added login feature”. Now switch back to the `master` or `main` branch and verify whether the login feature changes are visible or not. Merge the `feature-login` branch into the main branch and confirm the changes are merged successfully. Finally, delete the branch using the safe delete option.

Git Stash Assignment

Inside the existing repository, modify the `app.txt` file by adding the line “Temporary work”. Do not commit the changes. Check the Git status and observe the modified file. Now use the Git stash command to temporarily save the changes. Verify that the working directory becomes clean and confirm the temporary changes are removed from the file. After verification, restore the stashed changes using the appropriate Git stash command and confirm that the changes are restored successfully inside the file.

Git Reset Assignment

Create a new file called `reset.txt` and add multiple versions into the file. First, add “Version1” and commit it. Then add “Version2” and create another commit. Finally, add “Version3” and create the third commit. Check the commit history using `git log --oneline`. Perform a soft reset to remove the latest commit while keeping the

changes in the staging area. Verify the status using `git status`. Next, perform a mixed reset to remove the commit and unstage the changes. Again verify the status. Finally, perform a hard reset to permanently remove the commit and changes from the working directory. Verify the remaining commit history and explain the difference between soft, mixed, and hard reset.

Git Revert Assignment

Modify the `app.txt` file by adding the line “Buggy code added”. Commit the changes with the message “Bug introduced”. Check the commit history and identify the commit ID of the buggy commit. Use the Git revert command to safely undo the changes without deleting the commit history. After reverting, verify that Git creates a new commit automatically and confirm that the buggy code is removed from the file. Explain why revert is considered safer in production environments compared to reset.

Git Cherry-pick Assignment

Create a new branch called `bugfix` and switch into it. Inside the branch, create a file called `fix.txt` and add the text “Critical bug fixed”. Commit the changes with the message “Critical bug fixed”. Check the commit history and copy the commit ID of the bug-fix commit. Switch back to the main branch and use the Git cherry-pick command to copy only the selected commit into the main branch. Verify that only the required commit is copied without merging the entire branch. Explain real-time scenarios where cherry-pick is useful in production environments.

Scenario-Based Assignment

Assume you are working on a feature branch and suddenly receive an urgent request to fix a production issue. You currently have unfinished local changes in your working directory. First, save the unfinished work using Git stash. Create a separate hotfix branch and add the production fix inside that branch. Commit the changes and merge the hotfix branch into the main branch. After successful merging, delete the hotfix branch. Finally, restore your unfinished work from stash and continue development.

Explain how stash, branching, merge, and branch deletion help developers in real-time DevOps projects.